Manual Part

2) Mutation Analysis

Given this code:

```
public boolean discountAllowed(int purchases)
{
    if (purchases >= 0 && purchases > 5)
        return true;
    else
        return false;
}
```

By assuming discountAllowed is written in a class called Discount,
We'll write these next two tests.

Part a – Writing Unit Tests

```
@Test
void validDiscount()
{
        Discount discount = new Discount();
        assertTrue(discount.discountAllowed(10));
}

@Test
void invalidDiscount()
{
        Discount discount = new Discount();
        assertFalse(discount.discountAllowed(1));
}
```

Part b – Applying Mutations

M1:

```
public boolean M1(int purchases)
{
    if (purchases >= 0 && purchases >= 5)
        return true;
    else
        return false;
}
```

Result - The mutant survives.

Explanation:

Our test cases checked for the inputs 10 and 1,

For the input 10 both the original code and the mutant M1 return true.

For the input 1 both the original code and the mutant M1 return false.

M2:

```
public boolean M2(int purchases)
{
    if (purchases >= 0 && purchases < 5)
        return true;
    else
        return false;
}
```

Result – The mutant killed.

Explanation:

For the input 10 the original code returns true and the mutant M2 returns false , Hence the test validDiscount() fails.

M3:

```
public boolean M3(int purchases)
{
    if (purchases >= 0 && purchases > 5)
        return false;
    else
        return false;
}
```

Result – The mutant killed.

Explanation:

For the input 10 the original code returns true and the mutant M3 returns false , Hence the test validDiscount() fails.

M4:

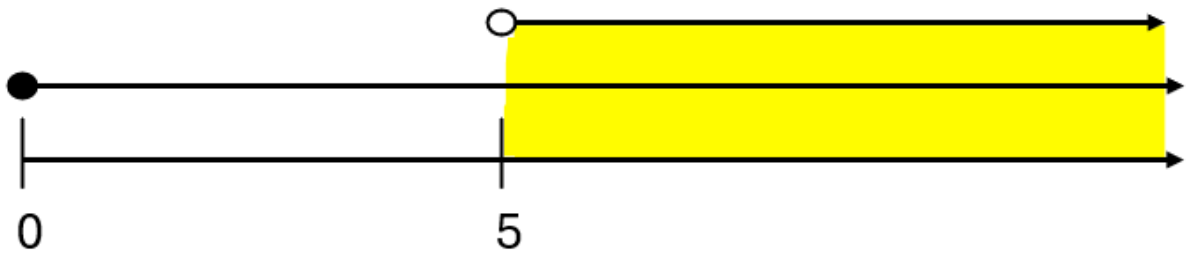
```
public boolean M4(int purchases)
{
    if (purchases > 5)
        return true;
    else
        return false;
}
```

Result – The mutant is equivalent.

Part c – Equivalent Mutant

Explanation:

Our test cases checked for the inputs 10 and 1,
For the input 10 both the original code and the mutant M4 return true.
For the input 1 both the original code and the mutant M4 return false.
So far the mutant survives meaning that we got the same results for the Tests we provided but potentially there are existing tests which we didn't write that could kill the mutant.
Let's explain why there are no such tests to get the mutant M4 killed.
The only condition that altered is from the original code is from (purchases >=0 && purchases >5) to (purchases >5).
But as you can see here:



Mathematically, The conditions
(purchases >=0 && purchases >5) and (purchases > 5) are equivalent!
And hence the mutant M4 is equivalent to the original code, meaning
It will provide the same results to the original code for every input,
Therefore there could not be any possible tests that can kill the mutant,
Hence the mutant not only survives, It's equivalent.

Part d – Calculate the Mutation Score

$$Mutation\ Score = \frac{Killed\ Mutants}{Total\ Mutants - EquivalentMutants} = \frac{2}{4-1} = \frac{2}{3}$$

$$\Rightarrow\ Mutation\ Score = 66.\overline{6}\%$$

3) AI question

AI response using Gemini Pro:

To guarantee a 100% mutation score, you actually have to do two things: select optimal boundary test cases, and **refactor the code itself**.

With the code exactly as it is written right now, you likely **cannot** achieve a 100% mutation score, regardless of which test cases you choose. Here is why, and how to fix it.

The Problem: Redundant Logic
Look closely at your condition:
if (purchases >= 0 && purchases > 5)

If an integer is strictly greater than 5 (i.e., 6, 7, 8...), it is mathematically impossible for it *not* to be greater than or equal to 0. The first half of the condition (purchases >= 0) is logically redundant.

Why This Ruins Your Mutation Score (Equivalent Mutants)
Mutation testing tools (like PIT for Java) work by changing your code slightly to see if your tests fail. If the tool removes the redundant condition, it creates a mutant that looks like this:
// Mutant code
if (purchases > 5)

This mutant behaves **identically** to your original code for every single integer in existence. Because the output never changes, no test case can ever fail against it. This is known as an **equivalent mutant**.

Because your tests cannot kill equivalent mutants, they survive, and your mutation score will be stuck below 100%.

The Solution: How to Guarantee 100%
To get a 100% mutation score, you must eliminate the redundancy and pick test cases right on the boundary.

Step 1: Refactor the Code
Remove the dead logic so mutation tools don't generate equivalent mutants. You can also simplify the if/else block into a single return statement.

```
public boolean discountAllowed(int purchases) {
   return purchases > 5;
}
```

Step 2: Pick the Best Test Cases
You are allowed one true test case and one false test case. To kill all possible remaining mutants (like changing > to >=, <, ==, or !=), you must use **boundary values**.
The boundary for purchases > 5 is between 5 and 6.

- **Test Case 1 (False):** discountAllowed(5) -> Asserts false
- **Test Case 2 (True):** discountAllowed(6) -> Asserts true

Why these exact numbers? If a mutator changes your code to purchases >= 5, your 5 test case will catch it (it expects false, but the mutant returns true). If a mutator changes your code to purchases > 6, your 6 test case will catch it (it expects true, but the mutant returns false).

By combining the simplified code with tight boundary tests, you guarantee that any change to the operator or the constant will be caught, yielding a 100% mutation score.

Errors/Ambiguities indications

Error

The Gemini Pro claims that "Because your tests cannot kill equivalent mutants, they survive, and your mutation score will be stuck below 100%".
In order to refute this claim, I'll now provide 2 tests case
(one true and one false) and an equivalent mutant, using the mutants that provided in HW1 (M1,M2,M3,M4).

Assuming these are my unit tests:

```
@Test
void validDiscount()
{
        Discount discount = new Discount();
        assertTrue(discount.discountAllowed(6));
}

@Test
void invalidDiscount()
{
        Discount discount = new Discount();
        assertFalse(discount.discountAllowed(5));
}
```

Hence:
M1 is killed (invalidDiscount() fails).
M2 is killed (validDiscount() fails).
M3 is killed (validDiscount() fails).
M4 is equivalent.

Therefore:

$$Mutation\ Score = \frac{Killed\ Mutants}{Total\ Mutants - EquivalentMutants} = \frac{3}{4-1} = \frac{3}{3} = 1$$

$$\Rightarrow Mutation\ Score = 100\%$$

Error and Ambiguity

The Gemini Pro suggested to refactor the code so no equivalent mutants will be generated in order to reach 100% mutation score.

This is false for two reasons:

1) Error - We showed earlier that it's possible to reach 100% mutation score even with equivalent mutants.

2) Ambiguity – In the QA world we get a code and our job is to write tests for it and not refactor it. Refactoring the code changes the system we suppose to test which contradicts the fundamental principles of software testing.